

# 工作坊讲义：DM0.5 与 DW0.5 在 SO-101 上的推理、微调与部署

## 目录

|                              |           |
|------------------------------|-----------|
| <b>1 今天要做什么</b>              | <b>2</b>  |
| 1.1 三类资源 . . . . .           | 2         |
| 1.2 一天的路线 . . . . .          | 3         |
| 1.3 本节小结 . . . . .           | 4         |
| <b>2 Dexbotic：框架与安装</b>      | <b>4</b>  |
| 2.1 Dexbotic 是什么 . . . . .   | 4         |
| 2.2 三步安装 . . . . .           | 4         |
| 2.3 本节小结 . . . . .           | 5         |
| <b>3 星禾套件：SO-101 真机与仿真器</b>  | <b>5</b>  |
| 3.1 机械臂 . . . . .            | 5         |
| 3.2 仿真器 . . . . .            | 5         |
| 3.3 一个接口驱动两种机器人 . . . . .    | 6         |
| 3.4 动手：让仿真里的机械臂动一动 . . . . . | 6         |
| 3.5 本节小结 . . . . .           | 7         |
| <b>4 DM0.5：原理与仿真部署</b>       | <b>7</b>  |
| 4.1 DM0.5 是什么 . . . . .      | 7         |
| 4.2 推理服务与控制循环 . . . . .      | 8         |
| 4.3 动手：在仿真里抓放 . . . . .      | 8         |
| 4.4 本节小结 . . . . .           | 9         |
| <b>5 DW0.5：原理与仿真部署</b>       | <b>10</b> |
| 5.1 DW0.5 是什么 . . . . .      | 10        |
| 5.2 怎样判断它“真的在看动作” . . . . .  | 10        |
| 5.3 动手：推演未来 . . . . .        | 10        |
| 5.4 本节小结 . . . . .           | 12        |

1 今天要做什么2

6 数据集12

6.1 两份数据12

6.2 下载与转换12

6.3 本节小结13

7 拓展：DM0.5 单卡微调13

7.1 LoRA 在 DM0.5 上训什么13

7.2 现场配方13

7.3 动手：训练并部署自己的模型14

7.4 本节小结15

8 拓展：DM0.5 真机部署15

8.1 同一个控制循环15

8.2 上机前准备15

8.3 本节小结16

9 本讲小结16

附录 A 版本16

参考文献17

这份讲义配合 Xbotics 与原力灵机联合举办的一天工作坊。一天里我们只做一件事：把两个开源的具身基础模型真正用到一台机械臂上。一个是视觉-语言-动作模型 (Vision-Language-Action, VLA) DM0.5, 看画面、听指令、出动作；一个是世界模型 DW0.5, 给它一串动作，它画出照这串动作做下去会看到的画面。平台是 SO-101 单臂机械臂和它的仿真器，任务是把桌上的物体抓起来放进料箱。

讲义自成一体，不要求读过别材料；会用 Linux 命令行和基本的 Python 即可。所有命令都能照着敲，所有数字都来自同一套公开代码和公开权重的实测。

# 1 今天要做什么

## 1.1 三类资源

整个工作坊只用到三类资源：

| 资源    | 链接                                                                                                |
|-------|---------------------------------------------------------------------------------------------------|
| 讲义    | 就是这份文档，也在教学仓的 lecture/pdf/ 下                                                                      |
| 代码    | <a href="#">教学仓 Xbotics-Dexmal-Workshop</a>                                                       |
| 数据与权重 | <a href="#">DM0.5 权重</a> 、 <a href="#">DW0.5 权重</a> 、 <a href="#">仿真数据</a> 、 <a href="#">真机数据</a> |

代码只有教学仓一个，装它的时候会自动从 GitHub 取来三个依赖：模型框架 Dexbotic、SO-101 仿真器、机器人接口 LeRobot，不需要分别下载、分别安装。权重和数据由教学仓里的下载命令取回，权重上午就要用，数据到第 6 节才用。图 1 画出了它们的关系。

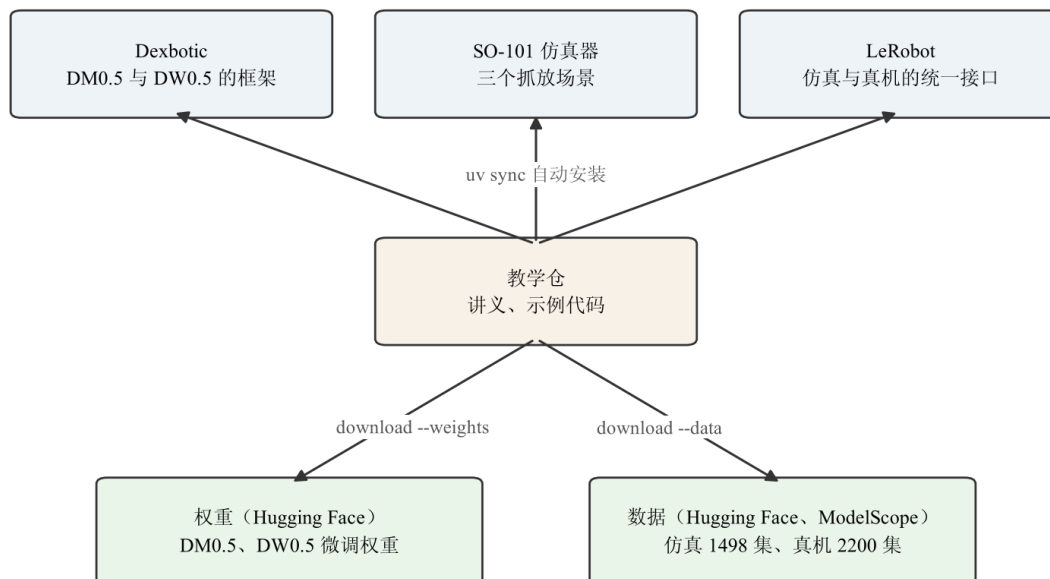


图 1: 教学仓与它自动安装的三个依赖，以及权重和数据的来处

## 1.2 一天的路线

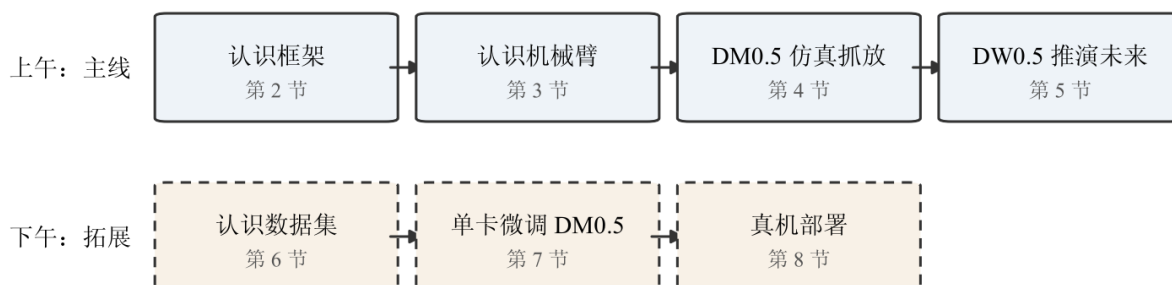


图 2: 今天的路线：先认识框架和机械臂，再用发布的模型在仿真里推理；训练与真机部署是拓展内容

图 2 是一天的路线。上午认识工具：先装好 Dexbotic 框架（第 2 节），再认识星禾套件里的 SO-101 机械臂和它的仿真器，让仿真里的机械臂动一动（第 3 节）。然后认识两个模型，并用在 SO-101 数据上微调好的权重在仿真里推理：DM0.5 抓放（第 4 节），DW0.5 推演未来（第 5 节）。下午是拓展内容：了解训练数据（第 6 节），在一张卡上微调一次 DM0.5 并部署自己训出的模型（第 7 节），最后把同一套代码接到真机上（第 8 节）。

### 1.3 本节小结

今天的主线是“认识工具、用发布的模型推理”，训练和真机是拓展。要记住的只有三类资源：讲义、代码（教学仓）、数据与权重。

## 2 Dexbotic: 框架与安装

### 2.1 Dexbotic 是什么

Dexbotic 是原力灵机开源的视觉-语言-动作模型工具箱 [1]。它要解决的问题是：各家的 VLA 模型用着不同的框架、不同的数据格式，想比较或复现几个模型，就得配好几套环境。Dexbotic 把它们统一到同一个代码库里，有三个做法贯穿始终。

第一，**模型统一拆成两部分**：视觉-语言主干和动作专家。主干由视觉编码器和大语言模型组成，负责看图、读指令；动作专家接在主干后面，负责生成动作。不同的 VLA 模型，差别主要在动作专家是什么结构。

第二，**实验以脚本为中心**。每个实验是一个 Python 配置文件，只写与默认配置不同的那几项；训练和推理服务都从这个文件启动。本工作坊的 DM0.5 和 DW0.5 各有一个这样的配置，放在 `dexbotic.so101` 里。

第三，**统一的数据格式 Dexdata**。不同机器人的数据都转成同一种格式：每一集一个 jsonl 标注文件，逐帧记录关节状态、动作和指令，画面仍然引用原来的视频文件。第 6 节会亲手做一次这种转换。

DM0.5 和 DW0.5 都在 Dexbotic 里：本工作坊用的版本把原力灵机的 DM0.5 工具箱与 DW0.5 世界模型 [2][3] 合并进同一个框架，并加上了 SO-101 这台机器人的数据转换、训练与评测。

### 2.2 三步安装

需要一张 32 GB 显存的 NVIDIA 显卡（现场是 RTX 5090）。先确认驱动够新：运行 `nvidia-smi`，右上角的 CUDA Version 不低于 12.8 即可。再装好 Python 项目管理工具 `uv`：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

然后三步：

```
# 第一步：装仓库（一个环境装好 Dexbotic、仿真器与 LeRobot）
git clone https://github.com/Xbotics-Embodied-AI-club/Xbotics-Dexmal-Workshop.git
cd Xbotics-Dexmal-Workshop
uv sync
```

```
# 第二步：下权重（两份微调权重，加上 DWO.5 推理要用的基座组件）
uv run python -m dexmal_workshop.download --weights

# 第三步：下数据（第 6 节才用，可以先跳过）
uv run python -m dexmal_workshop.download --data
```

uv sync 把 Dexbotic、仿真器和 LeRobot 从各自的 GitHub 仓按固定版本装进同一个环境，同时装好 PyTorch 等依赖；这几个项目之间的版本冲突教学仓已经处理好，学员不需要管。权重和数据都放在 `~/so101_workspace/` 下，设环境变量 `SO101_ROOT` 可以换到别处。现场的机器已经提前装好、下好，三步都是课后在自己机器上复现时才需要。

之后所有命令都在教学仓目录下用 `uv run` 执行，它会自动使用刚才装好的环境。装好之后，Dexbotic、仿真器和本工作坊的代码都可以直接 `import`，第 3.4 节就会用到。

## 2.3 本节小结

Dexbotic 把 VLA 模型统一成“主干 + 动作专家”，用脚本管理实验，用 Dexdata 统一数据。安装只有三步：装仓库、下权重、下数据。

# 3 星禾套件：SO-101 真机与仿真器

## 3.1 机械臂

【待填：星禾套件的介绍——套件组成、与 SO-101 的关系、现场设备。等活动组提供资料后补。】

SO-101 是一台桌面级单臂机械臂，五个关节加一个夹爪。它的状态是六个数，依次是 `shoulder_pan`（底座旋转）、`shoulder_lift`（大臂俯仰）、`elbow_flex`（肘）、`wrist_flex`（腕俯仰）、`wrist_roll`（腕旋转）五个关节的角度（单位：度），加 `gripper` 夹爪开合（0 到 100 的行程百分比，0 为合拢）。控制它就是给这六个数的目标值。机械臂上装两路相机：一路从上方看整个桌面（顶视），一路装在手腕上看夹爪前方（腕部），分辨率都是 640×480，每秒 30 帧。

## 3.2 仿真器

仿真器基于 ManiSkill3[4] 搭建，按真机标定，做了三个抓放场景。命令里用第二列的名字选场景：

| 场景      | 命令里的名字     | 难点               |
|---------|------------|------------------|
| 4 cm 方块 | cube40     | 基础场景             |
| 2 cm 方块 | cube20     | 物体小，夹持余量少，稍一偏就滑出 |
| 罐子      | cylinder40 | 圆面，接触点少，夹歪了容易转   |

每一局里物体和料箱的位置随机摆放。成功与否由仿真器自己判定：物体的水平位置落在料箱开口范围内、机械臂没有碰着物体、机械臂没有碰着料箱、机械臂已经静止，四条同时满足才算。

### 3.3 一个接口驱动两种机器人

真机和仿真器看上去完全不同，但对控制程序来说，它们是同一种东西：一台“机器人”。我们用 LeRobot[5] 的机器人接口来表达这件事，它只有两个核心操作：

| 操作                               | 作用     | 内容                                                                                       |
|----------------------------------|--------|------------------------------------------------------------------------------------------|
| <code>get_observation()</code>   | 读一帧观测  | 六个关节读数（键名如 <code>shoulder_pan.pos</code> ），外加 <code>top</code> 、 <code>wrist</code> 两路画面 |
| <code>send_action(action)</code> | 下发一步动作 | 六个关节的目标值，键名同上                                                                            |

仿真器和真机都实现了这两个操作，而且键名、单位完全一样。于是后面的控制程序只写一份：在仿真里跑通的代码，换一个参数就能驱动真机。这里只把 LeRobot 当成机器人的硬件接口来用，模型和推理服务都来自 Dexbotic。

### 3.4 动手：让仿真里的机械臂动一动

```
uv run python -m dexmal_workshop.hello_sim
```

这个脚本造出一台仿真的 SO-101，读一帧观测，然后在 3 秒里让手腕左右转一个来回、夹爪开合两次，把两路画面存成录像 `hello_sim.mp4`。自己在 Python 里试，只要几行（用 `uv run python` 进入交互环境）：

```
from dexmal_workshop.robots import make_sim

robot = make_sim("cube40")          # 一台由仿真扮演的 SO-101
robot.connect()
obs = robot.get_observation()        # 六个关节读数 + 两路画面
print({k: round(obs[k], 1) for k in robot.action_features})
target = {k: obs[k] for k in robot.action_features}
target["wrist_roll.pos"] += 30      # 手腕的目标转 30 度
for _ in range(30):                # 每次 send_action 只走一步 (1/30 秒)，连发
    ↪ 30 步
```

```
robot.send_action(target)
print(robot.get_observation()["wrist_roll.pos"])
robot.disconnect()
```

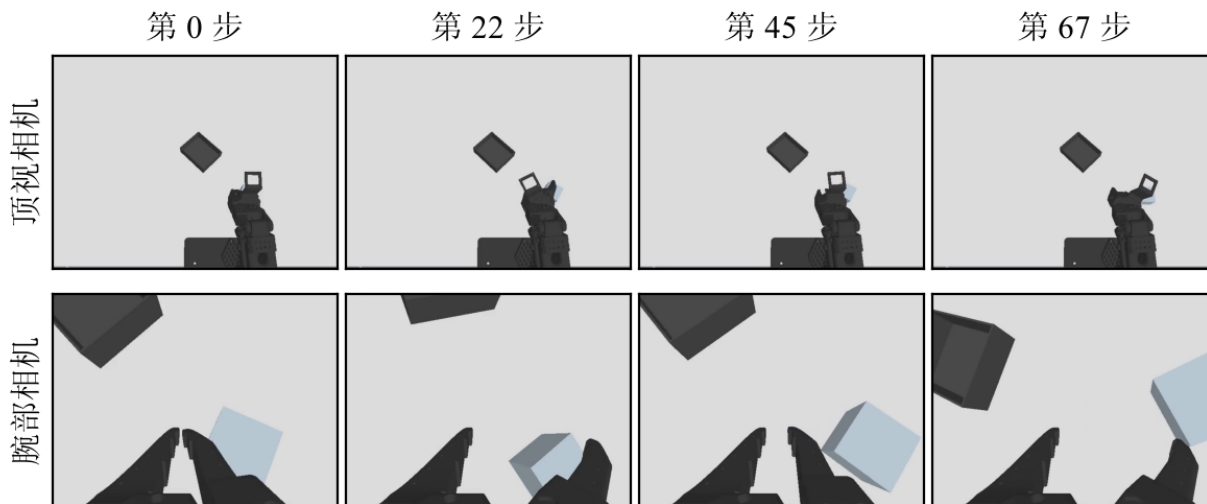


图 3: 仿真里的 SO-101 转动手腕、开合夹爪：上行为顶视相机，下行为腕部相机

图 3 是录像中的几帧。腕部相机随手腕一起转，所以下行画面整体旋转；夹爪张开又合上。运行时脚本还会打印六个关节的读数和这个场景的任务指令，可以对照 3.1 节看看每个数的含义。

### 3.5 本节小结

SO-101 的状态和动作都是六个数，两路相机分别看桌面和夹爪前方。仿真器与真机实现同一个 LeRobot 机器人接口，所以控制程序只写一份。

## 4 DM0.5: 原理与仿真部署

### 4.1 DM0.5 是什么

DM0.5 是原力灵机发布的视觉-语言-动作模型 [2]，约 61.5 亿参数，结构属于第 2.1 节说的“主干 + 动作专家”：

- **视觉-语言主干**基于 Gemma3，读入两路相机画面、当前关节状态和一句任务指令，算出一组中间表示；
- **动作专家**是一个专门生成动作的 Transformer，读主干的中间表示，一次给出接下来 50 步动作（一个动作块）。

动作专家用**流匹配**生成动作：从一段随机噪声出发，迭代 10 步，每一步都把这段“动作”往更合理的方向推一点，最后得到 50 步关节目标。每一步还要告诉动作专家“现在走到第几步了”，这个时间信息由每一层的时间调制层注入。

我们用 SO-101 的仿真与真机数据对 DM0.5 做了 LoRA 微调（第 7 节讲 LoRA），发布为 `Harrysunshine/so101-dm05-lora-sim-real-10task`。

## 4.2 推理服务与控制循环

DM0.5 以推理服务的形式运行：它监听一个网络端口，收到一帧观测就返回一个动作块。控制循环在另一个进程里，负责和机器人打交道：

1. 从机器人读一帧观测；
2. 把两路画面、六维关节状态和指令发给推理服务，拿回 50 步动作；
3. 依次下发其中的前 25 步；
4. 回到第 1 步，直到任务成功或达到步数上限（每局 500 步）。

为什么执行一半就重新请求？全部走完再看，中途物体被碰偏了也要很久才纠正；每走一步就问一次，推理次数又太多。25 步是两者之间的折中。

有三个参数必须和训练时一模一样，配错了都不报错，只会让成功率莫名其妙地低：

| 参数         | 要求                                                    | 配错的表现            |
|------------|-------------------------------------------------------|------------------|
| 相机顺序       | 1 号顶视、2 号腕部                                           | 模型把腕部当顶视看，动作方向乱  |
| 任务指令       | 与训练数据里的指令逐字相同，例如“Pick up a cube and place in the bin” | 模型没见过这句话，退化竞猜    |
| 动作是绝对值还是增量 | 服务返回的是绝对关节角，直接下发                                      | 再加一次当前状态，目标角约成两倍 |

最后一条值得多说一句。DM0.5 训练时学的是“相对当前关节角再转多少度”，但推理服务在返回前已经把当前状态加了回去，返回的就是绝对角。如果调用方以为拿到的是相对量、自己再加一次，目标角就变成了大约两倍。这类错误不报错，只表现为一个很低的成功率，很容易被误读成“模型没学会”。

## 4.3 动手：在仿真里抓放

先在一个终端起推理服务（第一次运行会下载 DM0.5 的基座权重，要几分钟）：

```
uv run python -m dexmal_workshop.serve
```

服务就绪后，在另一个终端跑 10 局 4 cm 方块，再给录像加上标注：

```
uv run python -m dexmal_workshop.rollout --robot sim --scenes cube40 --episodes 10
  ↳ --out out/rollout
uv run python -m dexmal_workshop.label_videos --rollout-dir out/rollout --out out/
  ↳ labeled
```

out/rollout/rollout\_dm05.json 里是成功局数，out/labeled/ 里是顶视与腕部并排、顶上写明成败的录像。图 4 是发布的模型在 2 cm 方块场景里的一局成功示例：它先把夹爪移到方块上方，下降合拢，再抬起平移到料箱上方。

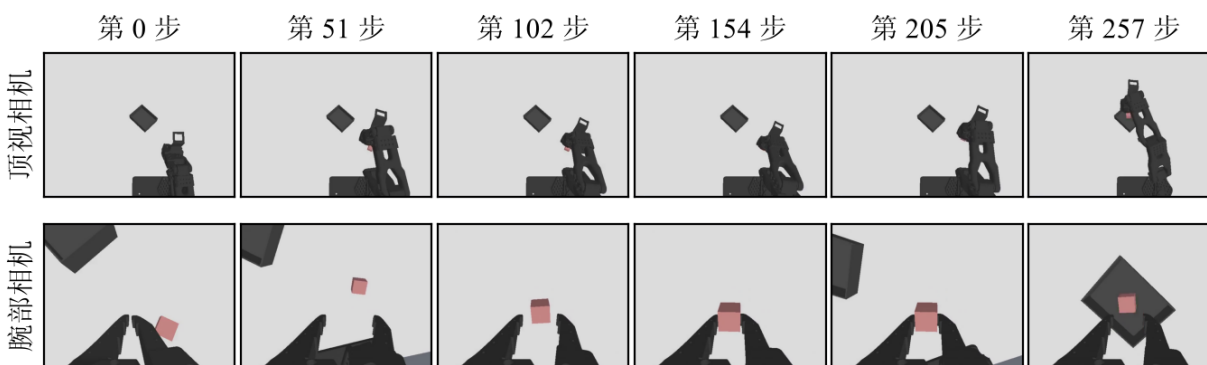


图 4: DM0.5 在仿真里抓放 2 cm 方块的一局：上行顶视相机，下行腕部相机

我们用同一条命令让发布的模型在三个场景各跑了 50 局（第  $i$  局用种子  $i$  摆放物体，`--seed` 默认从 0 开始，所以现场跑的 10 局就是这 50 局里的前 10 局）：

| 场景      | 成功    |
|---------|-------|
| 4 cm 方块 | 47/50 |
| 2 cm 方块 | 46/50 |
| 罐子      | 47/50 |

其中 4 cm 方块的前 10 局是 9 局成功，可以拿来和自己跑出的结果对照；换一台机器或一张显卡重跑，差一两局很正常。看录像时建议成功和失败各挑几局对照着看：失败的局多数是物体已经抓起来、却没有放进料箱，放置这一段是它相对的短板。

#### 4.4 本节小结

DM0.5 由 Gemma3 主干和流匹配动作专家组成，一次给出 50 步动作。部署时它是一个推理服务，控制循环每执行 25 步重新请求一次。相机顺序、指令原文、动作是绝对值还是增量，这三件事必须与训练一致。

## 5 DW0.5: 原理与仿真部署

### 5.1 DW0.5 是什么

DW0.5 是原力灵机发布的世界模型 [3]。普通的视频生成模型只看一张图和一句话就往下编；DW0.5 还要接收一串动作，要求生成的画面和这串动作对得上：给它“机械臂向左下方移动并合拢夹爪”，画出来的就应该是这个过程。

它的主干来自开源的 Wan2.2 视频生成模型 [6]，旁边接一个处理动作的扩散 Transformer，两者在每一层交换信息，用流匹配一起去噪（即第 4.1 节那种从噪声一步步推出结果的做法）。所以它既能同时生成未来的画面和动作，也能在给定动作的条件下只生成画面——本工作坊用的是后一种：拿一段真实轨迹的动作去驱动它，看它画出的未来与真实发生的画面有多像。

它每一轮接收 32 步动作、输出 9 帧画面，相邻两帧间隔 4 步。帧从 0 开始数：第 0 帧就是这一轮的起始画面，新画出来的是第 1 到第 8 帧。一轮画完，它把自己画的最后一帧当成下一轮的起始画面继续推。输入画面是三路拼成的一张图：上方一张大的顶视图，下方两张小的腕部图——SO-101 只有一路腕部相机，所以这一路用了两次，让画面布局和预训练时见过的一致。

### 5.2 怎样判断它“真的在看动作”

最直接的想法是：把生成的画面和真实发生的画面比一比，越像越好。常用的相似度指标是峰值信噪比 (PSNR)，单位分贝，越高越像；两段画面的 PSNR 差 4 dB，大致相当于逐像素误差差了 1.6 倍。

但只看这一个数有两个陷阱。第一，**什么都不预测也能拿高分**：桌面画面大部分是静止的背景，把起始画面原样复制若干遍，PSNR 也不会低。所以设一个参照：**复制起始帧**。第二，**画面在动，不等于按动作在动**：模型可能只学会了“机械臂一般会往物体那边移动”，不管给什么动作都画出差不多的运动。所以再设一个参照：**把同一串动作倒过来喂**，数值范围一样，只是时间顺序反了。

判据是：按真实动作推演的画面，平均 PSNR 要同时高于“复制起始帧”和“倒放动作”，并且逐条轨迹看，真实动作胜过倒放的要占多数。计分从第 1 帧开始：第 0 帧是交给模型的起始画面，“复制起始帧”在这一帧和真实画面逐像素相同，算进去会平白抬高这个参照。

### 5.3 动手：推演未来

DM0.5 的推理服务约占 12 GB 显存，DW0.5 推演约占 26 GB，一张 32 GB 的卡放不下两个，所以先在第一个终端按 Ctrl-C 停掉推理服务，再运行：

```
uv run python -m dexmal_workshop.imagine --rollout-dir out/rollout --out out/imagine
```

它读第 4.3 节跑出的轨迹：每一局都存下了逐帧的两路录像和逐步的关节状态，这些轨迹 DW0.5 训练时从没见过。喂给它的“真实动作”，是轨迹里每一步之后实际到达的关节状态。它从每个跑过的场景

取 2 条轨迹——按第 4.3 节的命令只跑了 4 cm 方块，所以就是 2 条——每条从起始画面起连推 3 轮、共 96 步。结果在 `out/imagen/dw05_sim_check.json`，`passed` 为真即判据通过。

我们用同样的命令在更多轨迹上做了检验：三个场景各取 4 条（发布的 DM0.5 在第 4.3 节那 50 局里存下的前 4 局，其中 2 条是失败局），共 12 条。

| 喂给模型的动作    | 与真实画面的平均 PSNR |
|------------|---------------|
| 真实动作       | 22.97 dB      |
| 复制起始帧（不预测） | 19.03 dB      |
| 倒放的动作      | 17.33 dB      |

12 条轨迹里，按真实动作推演的画面全部比倒放的更接近真实画面，平均比复制起始帧高出约 4 dB。

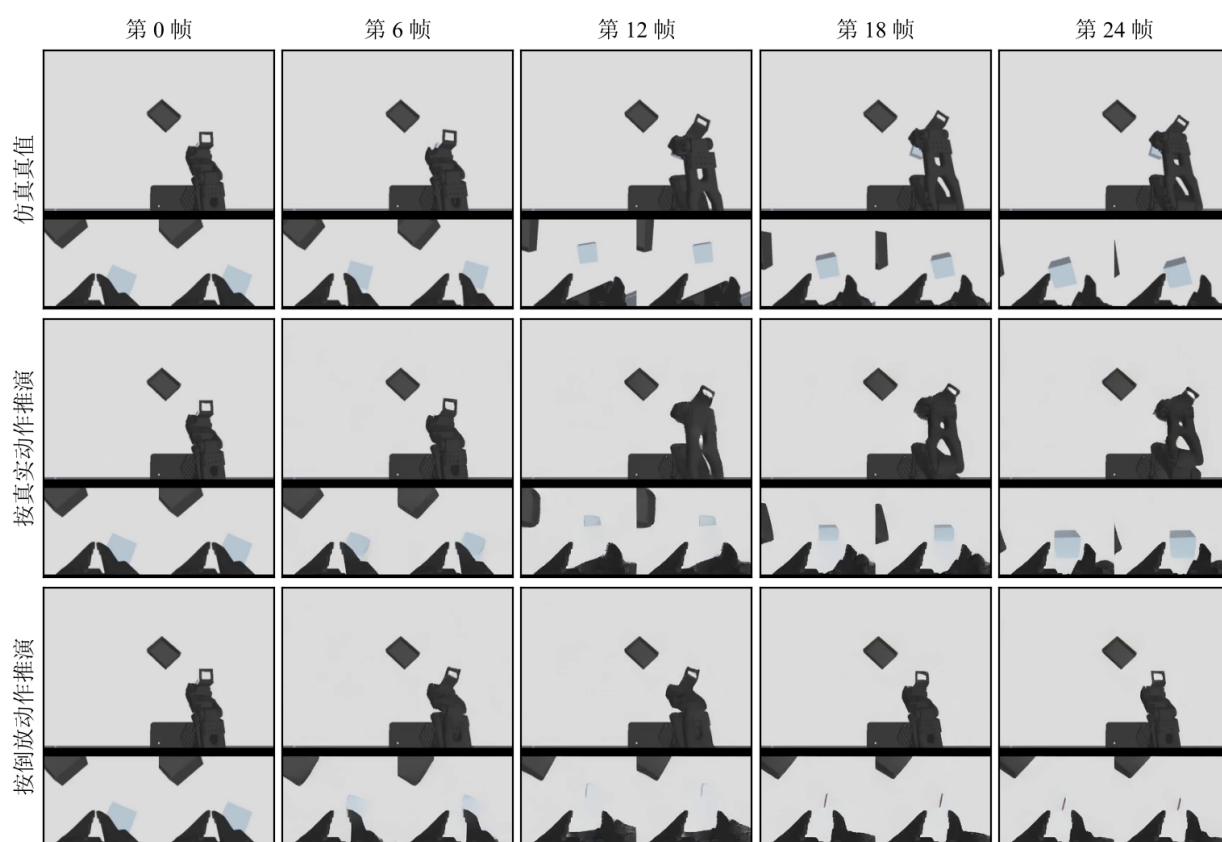


图 5: DW0.5 推演未来的三行对照：仿真真值、按真实动作推演、按倒放的同一串动作推演

图 5 是其中一条 4 cm 方块轨迹。三轮连推共 24 帧新画面，图中按总帧号标出五个时刻。中间一行和上面的真实画面几乎一致：手臂向前下方伸向方块的时机和姿态都对得上，腕部视角里方块出现在夹爪前方的位置也一样。下面一行给的是倒放的动作：手臂几乎停在原处，没有伸向方块，腕部视角里方块也不见了。

## 5.4 本节小结

DW0.5 在 Wan2.2 视频生成模型上加了动作条件，给一串动作就能画出未来。评价它要用两个参照：复制起始帧排除“什么都不预测”，倒放动作排除“不看动作”。

# 6 数据集

## 6.1 两份数据

DM0.5 与 DW0.5 用的是同一份训练数据，由两份公开数据集拼成，共 3698 集、约 128 万帧、10 个任务，全部 30 帧/秒、640×480、顶视加腕部两路相机：

| 来源 | 内容                | 规模     |
|----|-------------------|--------|
| 仿真 | 第 3.2 节三个场景的脚本化示教 | 1498 集 |
| 真机 | 遥操作采集，九个抓放与堆叠任务   | 2200 集 |

两份数据里有两个任务是重合的（抓方块、抓罐子），仿真独有“抓小方块”，真机独有另外七个（电池、橡皮、高尔夫球、药瓶、毛绒玩具、方块叠罐子、小方块叠大方块）。混在一起训练的理由很朴素：任务越多，模型见过的画面和指令变化就越多，越不容易只记住某一种。

两份数据都是 LeRobot 格式：一“集”是一次从摆好物体到任务结束的完整录制；画面按相机分别存成视频文件，许多集共用一个视频文件，另有表格文件逐帧记录关节状态、动作和时间戳。混合之前要逐项确认它们真的是“同一种数据”：分辨率、帧率、关节单位、夹爪的数值定义都一致。混进一份分辨率不同的数据，训练照样能跑、损失照样下降，只是模型学到的是两种互不相干的画面。

## 6.2 下载与转换

```
uv run python -m dexmal_workshop.download --data      # 两份数据集
uv run python -m dexmal_workshop.prepare_data         # 转成 Dexdata
```

下载命令把两份数据放到 `~/so101_workspace/datasets/`，每份都钉死了版本：数据换了版本，训练结果就不可比。转换命令把仿真三个场景和真机九个任务转进同一份 Dexdata：每一集一个 jsonl 文件，共 3698 个；每一行是一帧，记着这一帧的关节状态、动作、指令，以及两路画面在原视频里的位置。下面是仿真 4 cm 方块第 0 集的第一行（数字保留一位小数，省去了其余字段）：

```
{"images_1": {"type": "video", "url": "so101-sim-640-v2/cube40/videos/
↪ observation.images.top/chunk-000/file-000.mp4", "frame_idx": 0},
"images_2": {"type": "video", "url": "so101-sim-640-v2/cube40/videos/
↪ observation.images.wrist/chunk-000/file-000.mp4", "frame_idx": 0},
```

```
"prompt": "Pick up a cube and place in the bin",  
"state":  [-4.0, -103.1, 90.5, 64.0, -1.9, 0.4],  
"action": [-4.0, -103.0, 90.4, 64.1, -1.9, 0.5]}
```

`images_1` 是顶视、`images_2` 是腕部，这就是第 4.2 节说的相机顺序在数据里的样子；`frame_idx` 是这一帧在视频文件里的位置；`state` 与 `action` 都是六维，前五个是关节角度，最后一个为是夹爪开合。

仿真和真机进同一份，是因为它们是同一台机器人、同样的六维动作、同样两路相机；不同任务靠每一帧的指令区分。

### 6.3 本节小结

训练数据是仿真 1498 集加真机 2200 集，共 10 个任务。两份数据格式相同、单位和定义一致，转成 Dexdata 后合在一起训练。

## 7 拓展：DM0.5 单卡微调

### 7.1 LoRA 在 DM0.5 上训什么

LoRA[7] 的做法是冻住原模型的权重，在每个选中的线性层旁边并一条“低秩旁路”——两个很窄的小矩阵相乘，参数量只是原层的零头——只训练旁路。我们在 DM0.5 的全部线性层上加 LoRA，另外把动作专家的输入输出投影和时间调制层整体放开训练，可训参数约 3.24 亿，占全部 61.5 亿参数的 5.27%。

为什么这几层要整体放开？输入输出投影直接连着机器人的动作空间：预训练时的机器人和 SO-101 的六维不是同一套关节，这几层要学的不是“微调一点”，而是换一套映射。

这也解释了为什么第 2.2 节下载的 DM0.5 权重只有几百 MB：发布的是训练出来的这部分参数，用的时候叠在原力灵机的 DM0.5 基座上。

### 7.2 现场配方

配方已经写在 `train_lora` 里，直接运行即可：

| 项目   | 取值                          |
|------|-----------------------------|
| 起点   | 原力灵机的 DM0.5 基座              |
| 训练数据 | 第 6 节转换好的全部 3698 集          |
| 训练步数 | 300                         |
| 学习率  | 前 30 步从 0 升到 $1e-4$ ，之后逐渐降低 |
| 显存   | 约 24 GB                     |
| 用时   | 约 70 分钟（在与现场同代的显卡上实测 68 分钟） |

### 7.3 动手：训练并部署自己的模型

训练前先停掉第 4、5 节的推理服务和推演，训练要用整张卡：

```
uv run python -m dexmal_workshop.train_lora --gpu 0
```

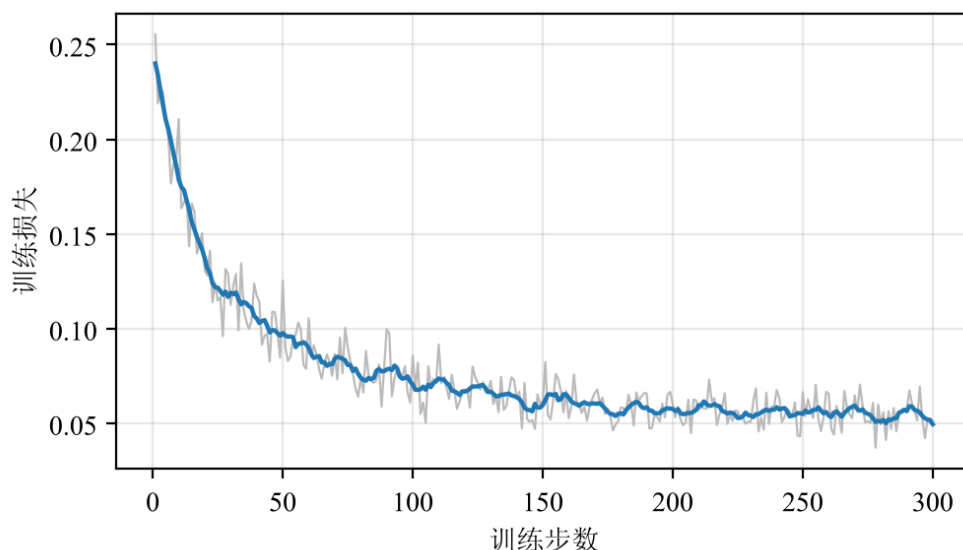


图 6: 单卡 LoRA 现场配方的训练损失：灰线为逐步损失，蓝线为滑动平均

图 6 是实测的训练损失：从约 0.26 很快降到 0.1 以下，最后稳定在约 0.05。

训练完，脚本自动用第 300 步的检查点（检查点就是训练途中保存下来的一份权重）做一次**开环自检**：从训练数据里取几个时刻，把那一刻的画面和状态喂给模型，只比较它预测的动作和数据里真实的后续动作差多少，不让它去驱动机器人；再和一个最朴素的参照比——“保持当前姿态不动”。结果在 `~/so101_workspace/runs/lora_single_gpu/openloop.json`。300 步只是入门，本讲实测模型的误差 (9.7) 与“保持不动” (9.4) 处在同一水平；要明显低于这个参照，需要更长的训练，发布的模型训了 5000 步。

最后把自己训出的模型部署到仿真里看一看——这和第 4.3 节是同一套命令，只是推理服务换成自己

的检查点:

```
uv run python -m dextral_workshop.serve --checkpoint ~/so101_workspace/runs/
↳ lora_single_gpu/checkpoint-300
uv run python -m dextral_workshop.rollout --robot sim --scenes cube40 --episodes 3
↳ --out out/my_model
```

300 步的模型还比较粗糙, 此前在 4 cm 方块上实测 20 局成功 2 局; 真机上建议仍用发布的权重。

## 7.4 本节小结

LoRA 只训 5.27% 的参数, 一张 32 GB 的卡就能微调 DM0.5。现场配方训 300 步约 70 分钟, 训完用同一套命令把自己的模型部署到仿真里。

# 8 拓展: DM0.5 真机部署

## 8.1 同一个控制循环

需要先说明: 本讲义的模型只在仿真中验证过, 真机抓放尚未实测, 这一节是探索性的内容。

真机部署用的是第 4 节同一个控制循环, 只把 `--robot sim` 换成 `--robot real`, 并告诉它串口、两路相机的设备号和这台臂的名字:

```
uv run python -m dextral_workshop.rollout --robot real --port /dev/ttyACM0 \
    --top-camera 0 --wrist-camera 2 --robot-id my_so101 \
    --prompt "Pick up a cube and place in the bin" --episodes 3 --out out/real
```

控制循环通过 LeRobot 的 SO-101 驱动读关节、下发目标, 关节读数用度, 和训练数据的单位一致。每一局开始前, 程序等操作者把物体摆好、手离开工作区后按回车; 每局结束时由操作者判定成败。真机上还多一道保险: 每一步每个关节相对当前读数最多只走 5 度 (或 5 个行程百分点), 模型给出离谱的目标时, 手臂也只会小步移动。

## 8.2 上机前准备

串口、相机号和标定都用 LeRobot 自带的命令 (装好教学仓就有):

```
uv run lerobot-find-port          # 插拔一次 USB, 找出机械臂的串口
uv run lerobot-find-cameras       # 列出相机和它们的编号, 分清哪个是顶视、哪个是腕部
uv run lerobot-calibrate --robot.type=so101_follower --robot.port=/dev/ttyACM0
↳ --robot.id=my_so101
```

标定时的 `--robot.id` 要和上面运行时的 `--robot-id` 相同，LeRobot 按这个名字找标定文件。

| 检查项           | 为什么                                          |
|---------------|----------------------------------------------|
| 机械臂已标定        | 标定零点不同，读数就会整体偏移，模型读到的姿态和训练时不一样               |
| 相机位置和视角贴近训练数据 | 顶视看全桌面、腕部看夹爪前方；视角差得多，模型没见过这种画面               |
| 相机编号对应正确      | 顶视和腕部接反不报错，模型把腕部当顶视看                         |
| 活动范围内清空杂物     | 第一次运行动作可能不对                                  |
| 准备好急停         | 在运行程序的终端按 <code>Ctrl-C</code> 停止，必要时直接断开舵机电源 |

8.3 本节小结

仿真与真机共用一个控制循环，真机只多了标定、相机和安全这几件事要自己确认。

9 本讲小结

这一讲把两个开源具身基础模型从“下载下来”走到“在机械臂上用起来”：

| 环节    | 关键做法                                             |
|-------|--------------------------------------------------|
| 安装    | 一个 <code>uv sync</code> 装好 Dexbotic、仿真器与 LeRobot |
| 机器人   | 仿真与真机实现同一个接口，控制循环只写一份                            |
| DM0.5 | 推理服务返回 50 步绝对角，每执行 25 步重新请求                      |
| DW0.5 | 给真实动作推演未来，用复制起始帧与倒放动作两个参照判断它是否在看动作               |
| 微调    | 单卡 LoRA 300 步，再用同一套命令把自己的模型部署到仿真里                |

贯穿全讲的一条经验是：**具身系统里最危险的错误不报错**。相机顺序、指令原文、动作口径、预处理，任何一处和训练不一致，程序都照常运行，只是结果安静地变差。对付它们的办法也是同一个：为每一个结论设一个参照，用能复查的结果文件说话。

一句话总结：把开源的 VLA 和世界模型用到自己的机器人上，模型之外最要紧的是接口——同一个接口驱动仿真和真机，每一处预处理都与训练一致，每一个结论都有参照。

附录 A 版本

代码（均在 GitHub 的 `Xbotics-Embodied-AI-club` 组织下）：

- 教学仓 `Xbotics-Dexmal-Workshop`

- 模型框架 dexbotic, 提交 3f2bc6b
- 仿真器 Xbotics-S0101-Sim, 提交 1510eee
- 机器人接口 lerobot, 提交 9381726

数据与权重:

- 仿真数据: Hugging Face Harrysunshine/so101-sim-pickplace-v2, 提交 92ca801c
- 真机数据: ModelScope zhuzhuangtian/so101-pick-place-tasks, 提交 f64493c4
- DM0.5 基座: Hugging Face Dexamal/DM05
- DW0.5 基座: Hugging Face Dexamal/DW05-Robotwin
- DW0.5 的视频生成主干: Hugging Face Wan-AI/Wan2.2-TI2V-5B
- 发布的 DM0.5: Hugging Face Harrysunshine/so101-dm05-lora-sim-real-10task
- 发布的 DW0.5: Hugging Face Harrysunshine/so101-dw05-sim-real-10task

DW0.5 推演结果、显存实测与训练损失的原始数据在教学仓 `lecture/data/` 下, 出图脚本在 `lecture/figures_src/`。

## 参考文献

- [1] Dexbotic Team. Dexbotic: open-source vision-language-action toolbox[EB/OL]. (2025)[2026-09-25]. <https://arxiv.org/abs/2510.23511>.
- [2] Dexamal. OpenDM: DM0.5 视觉-语言-动作模型开源代码与技术博客 [EB/OL]. [2026-09-25]. <https://github.com/dexamal/opendm>.
- [3] Dexamal. OpenDW: DW0.5 世界模型开源代码 [EB/OL]. [2026-09-25]. <https://github.com/dexamal/opendw>.
- [4] TAO S, XIANG F, SHUKLA A, et al. ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI[EB/OL]. (2024)[2026-09-25]. <https://arxiv.org/abs/2410.00425>.
- [5] CADENE R, ALIBERT S, SOARE A, et al. LeRobot: state-of-the-art machine learning for real-world robotics in PyTorch[EB/OL]. (2024)[2026-09-25]. <https://github.com/huggingface/lerobot>.
- [6] WAN TEAM, WANG A, AI B, et al. Wan: open and advanced large-scale video generative models[EB/OL]. (2025)[2026-09-25]. <https://arxiv.org/abs/2503.20314>.
- [7] HU E J, SHEN Y, WALLIS P, et al. LoRA: low-rank adaptation of large language models[EB/OL]. (2021)[2026-09-25]. <https://arxiv.org/abs/2106.09685>.